tifies those resident entries likely to be coalescible with the filled entry. Once the coalescing logic merges translations from a single LLC cache line, it then checks whether those resident entries can be further coalesced. In this way, a second TLB scan can actually be avoided, minimizing coalescing overheads.

4.3. CoLT-All Design and Implementation

Finally, CoLT-All coalesces into both set-associative L1/L2 TLBs and the superpage TLB. Its primary benefit over CoLT-SA and CoLT-FA is that it provides potentially the largest reach, at the expense of modifying both the set-associative and superpage TLB.

4.3.1. Overall operation. Figure 6 illustrates CoLT-All's operation when all the TLBs experience a miss. In step 1, the page walk has occurred and the coalescing hardware has determined the amount of contiguity present in the cache line. In then checks this contiguity to see how it compares to a threshold. If it is lower than a threshold (step 2), this means that the contiguity can be accommodated by the indexing scheme of the set-associative TLBs. For example, suppose the contiguity is three pages and we use an 8-set TLB with VPN[4-2] for indexing (allowing coalescing of up to four translations). In this case, the coalesced entry is allocated into the set-associative L1 and L2 TLBs. However the contiguity may be higher than the threshold and the amount that the set-associative TLBs can accommodate. In our example, the contiguity may be five. In this case, the entry is coalesced and brought into the superpage-TLB. At the same time, because the superpage-TLB is small, useful coalesced entries may be frequently evicted. Therefore, like CoLT-FA, we allocate an entry at this point into the L2 TLB as well. Unlike CoLT-FA however, our set-associative L2 TLB can now also handle coalesced entries (albeit with smaller levels of coalescing permissible by its choice of index bits). Therefore, CoLT-All brings in as much of this coalesced entry as possible into the L2 TLB, unlike CoLT-FA which brings just the requested translation. Finally, in step 4, the new allocated superpage entry may be coalesced with already-resident entries.

4.3.2. Lookup, replacement, invalidation, and attributes. While lookups operate similarly to CoLT-SA and CoLT-FA, the only difference is that it is possible for an entry to be resident in both the set-associative and fully-associative TLBs. While this occurs only rarely in practice, there are no correctness issues associated with this. Furthermore, there are no changes in replacement, invalidation, and attribute policies.

5. Methodology

We now detail the infrastructure and workloads used to quantify real-system contiguity and CoLT's effectiveness at leveraging this contiguity to eliminate TLB misses. Our analysis focuses on data pages since data references cause far more misses than instruction references [10, 27].

5.1. Real-System Characterizations of Page Allocation

5.1.1. Experimental platform and methodology. We use a system with a 64-bit Intel i7 processor, 64-entry L1 TLBs, and a 512-entry L2 TLB, a 32KB L1 cache, a 256KB L2 cache, a 4MB last-level cache (LLC), and 3GB of main memory. Furthermore, we run Fedora 15 (Linux 2.6.38).

To measure contiguity, we modify the kernel to scan the page table looking for instances of contiguous address translations. We walk the page table every five seconds, capturing contiguity changes through the benchmark run. Our original definition of contiguity is based only on page numbers; however, we now additionally require that contiguous translations must share the same page attributes and flags. While this eases the hardware implementation of CoLT by allowing for the same set of attribute bits per coalesced entry, contiguity would be even higher if this constraint were relaxed.

Benchmark	Suite	THS on L1/L2 MPMI	THS off L1/L2 MPMI
Mcf	Spec	56550/28600	95600/49230
Tigr	BioB.	19000/18150	26950/18860
Mummer	BioB.	12910/11450	14760/12970
CactusADM	Spec	6610/8140	8420/6930
Astar	Spec	8480/4660	17390/11240
Omnetpp	Spec	8410/2730	34040/8080
Xalancbmk	Spec	2670/2150	14120/2100
Povray	Spec	7010/630	7310/630
GemsFDTD	Spec	1300/620	8030/3620
Gobmk	Spec	710/410	1550/510
FastaProt	BioB.	460/300	610/300
Sjeng	Spec	1840/200	3860/440
Bzip2	Spec	4070/150	7120/270
Milc	Spec	120/90	3780/1820

Table 1: Summary of benchmarks used in our studies.

To study the effect of memory compaction, we use the Linux `defrag` flag. Enabling this flag triggers the memory compaction daemon both on page faults and as system background activity. Disabling this flag greatly reduces the number of times the memory compaction daemon runs. In tandem, we enable and disable THS to study the impact of superpaging. We also ensure that our system is realistically fragmented by using a machine that has already run a number of applications (eg. web browsers, network clients, office utilities) for two months. To further load the system, we run `memhog`, a memory fragmentation utility [12], with our workloads. We study scenarios where `memhog` fragments 25% and 50% (a highly fragmented system when combined with the other background activities) of the memory. In all, we study twelve system configurations. Due to space constraints, this paper focuses on:

1. THS on, normal memory compaction, no memhog: this is the current default setting for Linux.

2. THS off, normal memory compaction, no memhog: this shows contiguity *without* superpaging.

3. THS off, low memory compaction, no memhog: conservative case for contiguity because neither THS no memory compaction occur. The buddy allocator struggles to find contiguous physical blocks.

4. THS on, normal memory compaction, memhog: we test the effect of system load on the default Linux setting by assigning 25% and 50% of system memory to memhog.

5. THS off, normal memory compaction, memhog: shows the impact of fragmentation without superpaging.

5.1.2. Evaluation workloads. We study system contiguity on the Spec 2006 benchmarks [1] and bioninformatics workloads from Biobench [2] in Table 1. We run each of the workloads with their maximum data sets (for Spec, this corresponds to *Ref*) to completion. From the real-system runs, we use on-chip performance counters to track L1 and L2 TLB misses per million instructions (MPMI) when THS support is enabled and disabled. The benchmarks are ordered from highest to lowest THS on L2 TLB MPMIs. `Mcf`, `Tigr`, `Mummer`, `CactusADM`, and `Astar` see particularly high TLB MPMIs. While